# MOBİL PROGRAMLAMA PROJESİ TEKNİK RAPORU

## Öğrenci Asistanı: Akıllı Kampüs Cüzdanı, Ders & Görev Takip Sistemi

| | | |
|---|---|---|
| Öğrenci Adı Soyadı: Sinan Şamil Köylü | Öğrenci Numarası: 240211019 | Bölüm: Yazılım Mühendisliği |
| Ders: Mobil Programlama | Framework: Flutter 3.47+ & Dart | Veritabanı: SQLite (sqflite_ffi) + JSON |
| Mimari: Katmanlı Mimari (Clean Layered) | Durum Yönetimi: StatefulWidget + setState | Tasarım Dili: Modern Material 3 & FinTech UI |

## 1. Projenin Amacı, Vizyonu ve Çözdüğü Problemler

Öğrenci Asistanı, üniversite öğrencilerinin akademik hayatları boyunca karşılaştıkları iki hayati yönetim sorununu tek bir entegre mobil platformda çözmek üzere geliştirilmiştir: 1) Finansal Harcama & Bütçe Yönetimi (KYK bursu, aile harçlığı, yemekhane kartı dolumu, kitap/kırtasiye harcamaları) ve 2) Akademik Görev & Ders Takibi (ders ödevleri, laboratuvar raporları, proje teslimleri ve sınav takvimleri).

Öne Çıkan Mühendislik Hedefleri:
• %100 Yerel ve Güvenli Çalışma: Uygulama herhangi bir harici bulut sunucusuna veya internet bağlantısına bağımlı olmadan, cihaz üzerindeki ilişkisel SQLite motorunda tam performansla çalışır.
• FinTech Seviyesinde Kullanıcı Deneyimi: Bankacılık uygulamalarında kullanılan tek dokunuşla bakiye gizleme (••••••), canlı harcama oranı ilerleme çubuğu (progress bar) ve kategorik bütçe dağılım analizi.
• İlişkisel Veri Bütünlüğü: Kullanıcı tablosu ile işlemler ve görevler arasında yabancı anahtar (Foreign Key) ve basamaklı silme (ON DELETE CASCADE) güvencesi.
• JSON Veri Yedekleme & Kod İnceleme: Veritabanındaki tüm ilişkisel verilerin tek dokunuşla standart .json dosyasına dışa aktarılması (export) ve uygulama içi kod terminaliyle anlık doğrulanabilmesi.

## 2. Katmanlı Mimari ve Klasör Hiyerarşisi

Proje, endüstri standardı olan Clean Layered Architecture (Katmanlı Mimari) prensiplerine sadık kalınarak modüler bir yapıda inşa edilmiştir. Kod tabanı işlevlerine göre 4 ana katmana ayrılmıştır:

| Katman / Dizin | İçerik ve Görevi | Teknik Bileşenler ve Prensipler |
|---|---|---|
| **lib/models/** | UserModel, TransactionModel, TaskModel: Saf Dart sınıfları. Veritabanı tablolarındaki satırları nesneye, nesneleri Map ve JSON yapılarına dönüştürür. | toMap(), fromMap(), toJson(), fromJson(), copyWith() metotları. Immutable veri tasarımı. |
| **lib/database/** | DBHelper: Singleton desenli SQLite yöneticisi. Tablo kurulumu, migration ve SQL sorguları.<br>JsonService: Yerel dosya sistemi (File I/O) ve JSON serileştirme. | sqflite, sqflite_common_ffi, path_provider, PRAGMA foreign_keys = ON, aggregate SQL (SUM, COUNT). |
| **lib/screens/** | LoginScreen, RegisterScreen, HomeScreen: Uygulamanın tam ekran görünümleri. Form doğrulama, sekme navigasyonu ve kullanıcı etkileşimleri. | StatefulWidget yaşam döngüsü (initState, dispose, setState), Form GlobalKey, SharedPreferences. |
| **lib/widgets/** | CustomTextField: Özel animasyonlu ve odaklanmalı input.<br>AddTransactionModal & AddTaskModal: Alttan açılan şık veri giriş pencereleri. | showModalBottomSheet, InputDecoration stil kapsülleme, DatePicker, ChoiceChip/ActionChip. |

## 3. SQLite Veritabanı Mimarisi ve İlişkisel Veri Şeması

Uygulamada SQLite veritabanı ogrenci_asistani.db adıyla saklanmakta olup ilişkisel bütünlük PRAGMA foreign_keys = ON komutuyla garantiye alınmıştır. Bire-çok (1-to-N) ilişki mimarisi uygulanmıştır; bir kullanıcının birden fazla finans işlemi ve ders görevi olabilir. Kullanıcı silindiğinde basamaklı silme (CASCADE) sayesinde bağımlı kayıtlar da otomatik olarak temizlenir.

| Tablo Adı | Sütunlar (Kolonlar) & Tipleri | İlişki & Kısıtlamalar (Constraints) |
|---|---|---|
| **users (Kullanıcılar)** | • id: INTEGER PRIMARY KEY AUTOINCREMENT<br>• name: TEXT NOT NULL<br>• email: TEXT UNIQUE NOT NULL<br>• password: TEXT NOT NULL<br>• student_number: TEXT<br>• department: TEXT<br>• created_at: TEXT NOT NULL | Ana kullanıcı tablosu. E-posta adresi tekildir (UNIQUE). Diğer tablolar bu tablonun id alanına bağlanır. |
| **transactions (Finans İşlemleri)** | • id: INTEGER PRIMARY KEY AUTOINCREMENT<br>• user_id: INTEGER NOT NULL<br>• title: TEXT NOT NULL<br>• amount: REAL NOT NULL<br>• type: TEXT NOT NULL ('income' / 'expense')<br>• category: TEXT NOT NULL<br>• date: TEXT NOT NULL | FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE<br>Kullanıcı silindiğinde tüm gelir/gider kayıtları otomatik temizlenir. |
| **tasks (Ders & Ödevler)** | • id: INTEGER PRIMARY KEY AUTOINCREMENT<br>• user_id: INTEGER NOT NULL<br>• course_name: TEXT NOT NULL<br>• title: TEXT NOT NULL<br>• due_date: TEXT NOT NULL<br>• priority: TEXT NOT NULL ('Yüksek'/'Orta'/'Düşük')<br>• is_completed: INTEGER NOT NULL DEFAULT 0 | FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE<br>is_completed alanı SQLite'ta boolean temsili için 0 veya 1 olarak saklanır. |

### JSON Veri Yedekleme Formatı ve Serileştirme Standardı:

JsonService sınıfı, veritabanındaki tüm ilişkisel tabloları hiyerarşik bir JSON ağacına dönüştürerek getApplicationDocumentsDirectory()/ogrenci_asistani_yedek.json dosyasına kaydeder. Dosya formatı aşağıdaki gibidir:

```
{
  "proje_adi": "Öğrenci Asistanı - Flutter SQLite & JSON",
  "olusturucu": "Sinan Şamil Köylü",
  "yedekleme_tarihi": "2026-10-23T18:41:15",
  "toplam_kullanici": 2,
  "kullanici_verileri": [
    {
      "user": { "id": 2, "name": "Sinan Samil Koylu", "email": "koylusinansamil@gmail.com", ... },
      "transactions": [ { "id": 5, "title": "burs", "amount": 4000.0, "type": "income", ... } ],
      "tasks": [ { "id": 4, "course_name": "mobil programlama", "priority": "Yüksek", ... } ]
    }
  ]
}
```

## 4. Projede Kullanılan Tüm Widget'lar ve Kullanım Amaçları

Projeyi inşa ederken kullanıcı arayüzünü zenginleştirmek, Material 3 standartlarına tam uyum sağlamak ve yüksek performans elde etmek amacıyla Flutter SDK'sının güçlü widget kütüphanesinden yararlanılmıştır. Aşağıda projede aktif olarak kullanılan tüm widget'lar sınıflandırılarak ne için ve hangi amaçla kullanıldıkları ayrıntılı olarak dökümante edilmiştir:

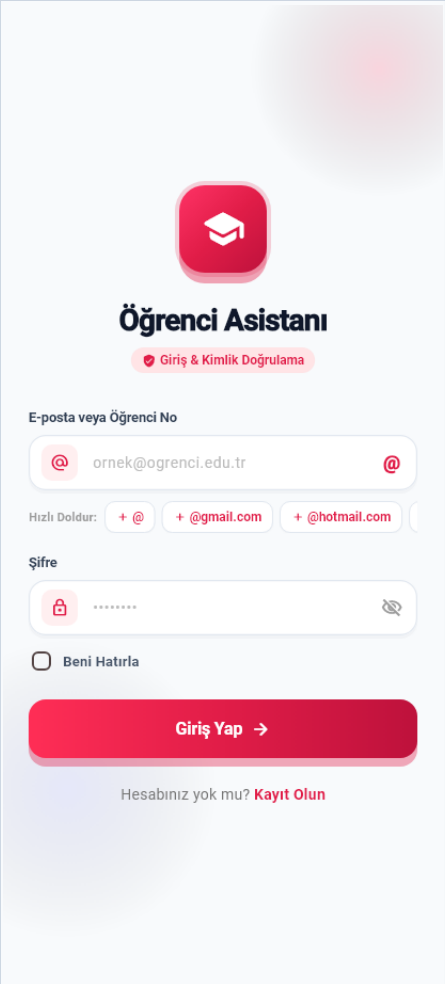
| Widget Adı | Kategori | Projedeki Kullanım Amacı ve Üstlendiği Fonksiyon |
|---|---|---|
| **Scaffold** | Düzen (Layout) | Tüm ekranların ana yapı iskelesidir. Sayfanın arka plan rengini, gövdesini (body), alt navigasyon barını ve yüzen aksiyon butonunu (FAB) tek merkezde birleştirir. |
| **SafeArea** | Düzen (Layout) | Mobil cihazların üst çentik (notch), durum çubuğu (status bar) ve alt ev göstergesi alanlarının içerikle çakışmasını engellemek için ekranı güvenli sınırlara hapseder. |
| **Stack & Positioned** | Düzen (Layout) | Öğeleri üst üste bindirmek için kullanılmıştır. Özellikle ana ekranın arka planındaki yumuşak radyal ışık parıltılarını (ambient glow efektleri) ve profil avatarının üzerindeki yeşil onay rozetini konumlandırmak için kullanılmıştır. |
| **Container** | Düzen & Stil | Kutusal modelleme. Kartların kenar yuvarlatmaları (BorderRadius), arka plan degrade geçişleri (LinearGradient), gölgelendirmeleri (BoxShadow) ve iç/dış boşlukları Container ile sağlanmıştır. |
| **Column & Row** | Düzen (Flex) | Arayüz elemanlarını dikey (Column) ve yatay (Row) eksende hizalamak. Bilgi kartları, buton grupları ve başlık satırlarının kusursuz yerleşimi için ana flex yöneticileridir. |
| **Expanded & Spacer** | Esneklik (Flex) | Satır ve sütunlar içinde kalan boşluğu dinamik olarak doldurmak; örneğin işlem kartlarında başlık metnini sola yaslarken tutar bilgisini en sağa itmek için kullanılmıştır. |
| **SizedBox & Padding** | Boşluklandırma | Bileşenler arasına sabit pixel cinsinden nefes alma boşlukları koymak ve ekran kenarlarından düzenli iç boşluk (örneğin yatayda 20px) bırakmak için kullanılmıştır. |
| **SingleChildScrollView** | Kaydırma (Scroll) | Form ekranlarında klavye açıldığında piksel taşma hatalarını (Bottom Overflow) önlemek ve profil/cüzdan sekmelerinin dikeyde rahatça kaydırılmasını sağlamak için kullanılmıştır. |
| **ListView.builder** | Dinamik Liste | SQLite'tan gelen yüzlerce harcama ve ödev kaydını sadece ekranda görünen kadar belleğe alarak (lazy loading) son derece akıcı ve bellek dostu bir listeleme performansı sunar. |
| **RefreshIndicator** | Etkileşim (Pull) | Kullanıcının ekranı yukarıdan aşağıya doğru parmağıyla çekerek SQLite veritabanındaki tüm bakiye, gelir, gider ve ödev verilerini anında yeniden sorgulaması için kullanılmıştır. |
| **Form & GlobalKey** | Form Yönetimi | Giriş, kayıt ve ekleme ekranlarındaki tüm girdi alanlarını tek bir grup altında toplar; `_formKey.currentState!.validate()` çağrısıyla tek seferde tüm validasyon kurallarını tetikler. |
| **TextFormField** | Girdi (Input) | E-posta, şifre, ad soyad, tutar ve ders adı gibi metinsel girdileri almak; düzenleme denetleyicisi (TextEditingController) ve doğrulama mantığı (validator) çalıştırmak için kullanılmıştır. |
| **DropdownButtonFormField** | Seçici (Picker) | Harcama ekleme modalında kategorileri (Yemek, Ulaşım, Burs, Kitap vb.) ikonlarıyla birlikte açılır bir menüden seçtirmek için kullanılmıştır. |
| **Checkbox** | Seçim (Toggle) | Ödev kartlarında görevin tamamlanıp tamamlanmadığını (is_completed 0/1) tek dokunuşla değiştirmek ve Login ekranında 'Beni Hatırla' tercihini yönetmek için kullanılmıştır. |
| **ChoiceChip & ActionChip** | Hızlı Çipler | Ödev öncelik seviyesi seçimi (Yüksek □, Orta □, Düşük □); emülatörde hızlı Türkçe karakter (ç, ğ, ı, ö, ş, ü) ve e-posta uzantısı (@gmail.com) ekleme butonları olarak kullanılmıştır. |
| **ElevatedButton** | Aksiyon Butonu | Giriş Yap, Kayıt Ol, İşlemi Kaydet ve JSON Dışa Aktar gibi yüksek öncelikli birincil (primary) eylemleri vurgulamak ve gölgeli tıklanma hissi vermek için kullanılmıştır. |
| **FloatingActionButton** | Yüzen Aksiyon | Ekranın sağ alt köşesinde havada duran, degrade renkli özel '+ İşlem Ekle' ve '+ Ödev Ekle' butonudur. Kullanıcının her an hızlı veri girmesini sağlar. |
| **Dismissible** | Kaydırarak Silme | İşlem veya ödev kartını parmakla sola/sağa kaydırarak veritabanından kalıcı olarak silmeyi sağlar. Arkada kırmızı çöp kutusu ikonu çıkarır. |
| **NavigationBar & Badge** | Menü & Rozet | Ekranın altında yüzen modern 3 sekmeli Material 3 navigasyon dock'u. Ödevler simgesinin üzerinde bekleyen ödev sayısını canlı kırmızı bir Badge (rozet) olarak gösterir. |
| **LinearProgressIndicator** | İlerleme Çubuğu | Aylık harcama/bütçe oranını (%2.5 harcandı) ve ödevlerin tamamlanma yüzdesini (1/4 Tamam) görsel olarak dolan renkli bir bar şeklinde sunmak için kullanılmıştır. |
| **showModalBottomSheet** | Açılır Modal | Yeni harcama ekleme, ödev ekleme ve harcama kategorik dağılım analizini sayfa değiştirmeden alttan şık yuvarlatılmış kartlar olarak açmak için kullanılmıştır. |
| **AlertDialog & SnackBar** | Diyalog & Toast | Güvenli oturum kapatma onayı, JSON yedekleme önizleme penceresi ve işlem başarı/hata bildirimlerini (yeşil/kırmızı kayan bildirimler) göstermek için kullanılmıştır. |

## 5. Kullanılan Kritik Property'ler (Özellikler) ve Tercih Nedenleri

Flutter'da bir widget'ın davranışı, görsel dili ve işletim sistemiyle etkileşimi parametreleri (properties) üzerinden yapılandırılır. Kullanıcı talebinde özellikle belirtildiği üzere, bu projede yer alan kritik property'ler, aldıkları değerler ve bu değerlerin neden seçildiği aşağıda derinlemesine incelenmiştir:

| Widget & Property Adı | Atanan Değer / Tip | Neden Kullanıldı? (Mühendislik Gerekçesi) |
|---|---|---|
| `TextFormField controller` | TextEditingController | Kullanıcının girdiği metne anlık olarak erişebilmek, emülatörde hızlı çip butonlarına basıldığında metin sonuna karakter eklemek ve SharedPreferences'tan 'Beni Hatırla' e-postasını doldurmak için zorunludur. |
| `TextFormField validator` | String? Function(String?) | Kullanıcı boş değer girmesin, e-posta formatı regex ile doğrulansın ve şifre en az 6 karakter olsun diye çalıştırılır. Hatalı durumlarda kırmızı uyarı mesajı döndürür. |
| `TextFormField obscureText` | bool (true / false) | Şifre karakterlerini gizleyerek yıldız veya nokta (••••••) olarak maskelemek için kullanılır. Göz ikonuna tıklandığında setState ile tersine çevrilerek (toggle) görünürlük sağlanır. |
| `TextFormField keyboardType` | TextInputType.number / emailAddress | Tutar girişinde kullanıcının karşısına sadece sayısal klavyenin açılmasını sağlayarak hatalı harf girişini engeller; e-posta alanında ise '@' işaretini içeren klavye tipini tetikler. |
| `InputDecoration prefixIcon & suffixIcon` | Widget (Container / Icon) | Kullanıcıya alanın işlevini görsel olarak anlatmak için sol tarafa renkli ikon kutusu yerleştirir; sağ tarafa ise şifre açma/kapama göz ikonunu konumlandırır. |
| `InputDecoration filled & fillColor` | true & Colors.white | Varsayılan şeffaf arka plan yerine temiz beyaz bir form alanı elde etmek ve açık gri arayüzde formların belirginleşmesini sağlamak için kullanılır. |
| `InputDecoration focusedBorder & errorBorder` | OutlineInputBorder(BorderSide) | Kullanıcı alana tıkladığında kenarlık rengini kurumsal gül kırmızısına (#E11D48), hatalı giriş yapıldığında ise canlı kırmızıya (#EF4444) dönüştürerek interaktif geri bildirim verir. |
| `BoxDecoration borderRadius: BorderRadius.circular` | Radius.circular(16 ile 28) | Keskin, sert ve köşeli tasarımlar yerine modern Material 3 standardı olan yumuşak kıvrımlı yüzeyler sunmak ve mobil dokunmatik hissini iyileştirmek için kullanılmıştır. |
| `BoxDecoration boxShadow: [BoxShadow]` | blurRadius: 14, offset: Offset(0, 6) | Kartların ve butonların z-ekseninde havada süzülüyormuş gibi 3 boyutlu derinlik (elevation) kazanması ve profesyonel FinTech havası oluşturması için düşük opaklıklı siyah gölge verilmiştir. |
| `BoxDecoration gradient: LinearGradient` | colors: [Color(1), Color(2)] | Düz, sıkıcı renkler yerine canlı ve enerjik ton geçişleri (Pembe-Kırmızı-Bordo) sağlamak; bakiye kartında ve butonlarda göz alıcı bir estetik oluşturmak için kullanılmıştır. |
| `SingleChildScrollView physics: AlwaysScrollableScrollPhysics` | ScrollPhysics | Ekrandaki içerik az olsa veya liste kısa kalsa dahi kullanıcının ekranı aşağı doğru çekebilmesini ve RefreshIndicator'ın her koşulda sorunsuz tetiklenebilmesini garanti altına alır. |
| `showModalBottomSheet isScrollControlled: true` | bool (true) | Modal pencerenin sadece ekranın yarısında sıkışıp kalmasını önler; formun kendi yüksekliği kadar veya ekranın %85'ine kadar uzamasına ve klavye açıldığında yukarı kaymasına izin verir. |
| `Dismissible confirmDismiss & onDismissed` | Future<bool> Function & VoidCallback | Kullanıcı kaydı yanlışlıkla kaydırıp silmesin diye silme onayı sağlar; onaylandığında ise SQLite veritabanındaki deleteTransaction veya deleteTask sorgusunu çalıştırır. |
| `NavigationBar selectedIndex & onDestinationSelected` | int & ValueChanged<int> | Alt menüdeki aktif sekmenin vurgulanmasını kontrol eder ve tıklandığında _currentTabIndex değerini güncelleyerek ilgili sayfa gövdesini yeniden çizer. |
| `LinearProgressIndicator minHeight & valueColor` | double (6-8) & AlwaysStoppedAnimation | İlerleme çubuğunun kalınlığını belirgin kılmak ve bütçe aşımına ya da kategoriye göre rengini dinamik olarak değiştirmek (yeşil, sarı, turuncu) için kullanılmıştır. |

## 6.1. Giriş Yap Ekranı (Login Screen)




### Ekran Amacı ve Yapılan İşlemler:

Kullanıcının kayıtlı e-posta ve şifresiyle sisteme güvenli giriş yapmasını sağlar. SharedPreferences entegrasyonuyla 'Beni Hatırla' tercihi yönetilir. Emülatör ve mobil klavyelerde veri girişini hızlandırmak için hazır e-posta tamamlama çipleri (+ @, + @gmail.com, + @hotmail.com) içerir.

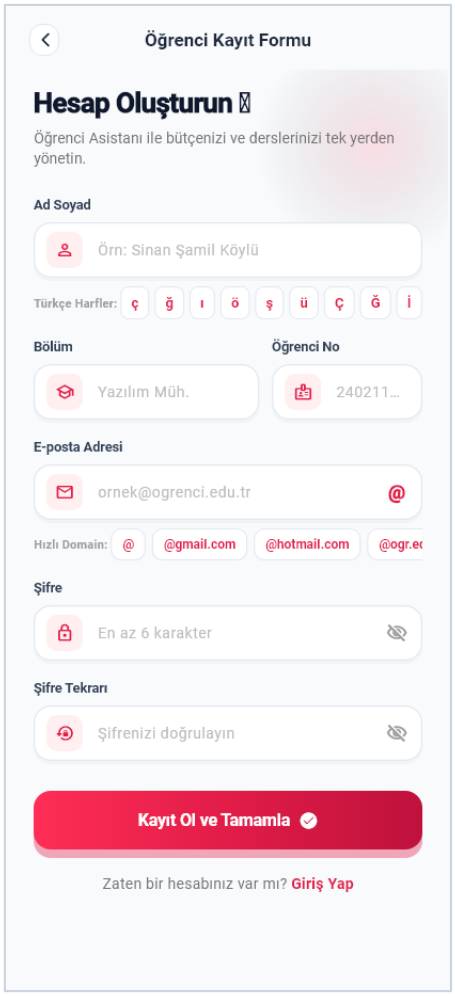
### Kullanılan Başlıca Widget'lar ve Rolleri:

| Widget | Kullanım Amacı |
|---|---|
| **Scaffold, SafeArea, SingleChildScrollView** | Klavye açıldığında taşmayı önleyen esnek ve güvenli sayfa yapısı. |
| **Form & GlobalKey** | E-posta ve şifre kurallarını tek tuşla doğrulayan form yöneticisi. |
| **CustomTextField** | Özel degrade ikonlu, gölgeli ve yuvarlatılmış e-posta ve şifre kutuları. |
| **Checkbox & CheckboxListTile** | 'Beni Hatırla' tercihini yakalayan ve SharedPreferences'a kaydeden seçim. |
| **ElevatedButton** | Degrade renkli, sağa ok ikonlu birincil 'Giriş Yap' butonu. |
| **ActionChip** | E-posta kutusuna tek tıkla @gmail.com veya @ ekleyen yardımcı çipler. |

### Öne Çıkan Property'ler ve Tercih Sebepleri:

| Property & Değeri | Neden Seçildi? (Teknik Gerekçe) |
|---|---|
| `obscureText: _obscurePassword` | Şifrenin gizlenmesini sağlar; göz butonuna basılınca tersine döner. |
| `keyboardType: emailAddress` | Klavyenin e-posta formatında (@ sembollü) açılmasını sağlar. |
| `validator: (value) => ...` | Boş geçilmezlik ve geçerli e-posta regex kontrolü yapar. |
| `borderRadius: BorderRadius.circular(16)` | Form giriş kutularına ve butonlara modern estetik hatlar verir. |

## 6.2. Kayıt Ol Ekranı (Register Screen)




### Ekran Amacı ve Yapılan İşlemler:

Yeni öğrencilerin sisteme hesap açmasını sağlar. Ad Soyad, Öğrenci No, Bölüm, E-posta ve Şifre bilgileri toplanır. SQLite üzerinde e-posta adresi benzersizlik (UNIQUE) kontrolünden geçirilir; mükerrer kayıtlar engellenir. Hızlı Türkçe karakter çipleri (ç, ğ, ı, ö, ş, ü) sunar.

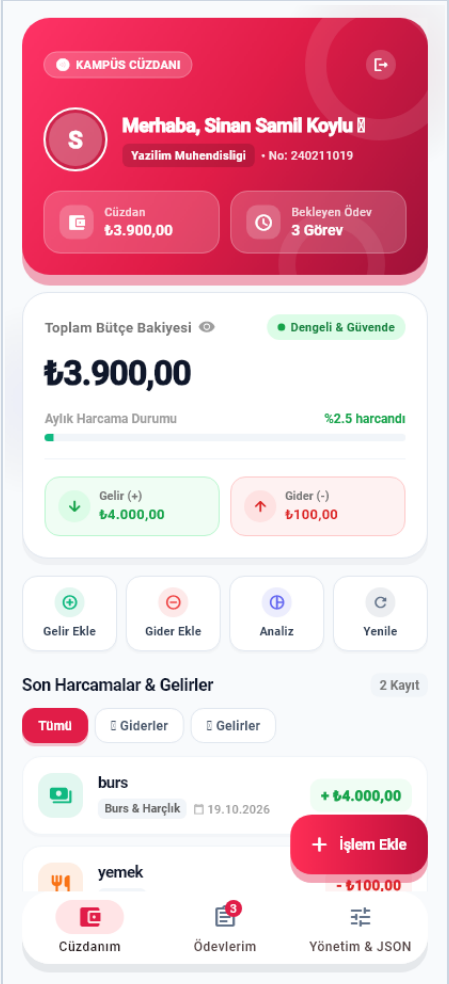
### Kullanılan Başlıca Widget'lar ve Rolleri:

| Widget | Kullanım Amacı |
|---|---|
| **Form & GlobalKey** | 5 farklı giriş kutusunun bütünsel doğrulanması. |
| **CustomTextField (5 Adet)** | Ad Soyad, Öğrenci No, Bölüm, E-posta ve Şifre için özelleştirilmiş inputlar. |
| **ActionChip (Türkçe Harfler)** | Emülatörde klavyede zor bulunan ç, ğ, ı, ö, ş, ü harflerini tek dokunuşla ekleme. |
| **ElevatedButton** | Formu doğrulayıp SQLite users tablosuna kaydeden 'Hesap Oluştur' butonu. |
| **SnackBar** | Kayıt başarılı veya mükerrer e-posta uyarısını kullanıcıya bildiren toast. |

### Öne Çıkan Property'ler ve Tercih Sebepleri:

| Property & Değeri | Neden Seçildi? (Teknik Gerekçe) |
|---|---|
| `conflictAlgorithm: abort` | SQLite'ta aynı e-posta varsa kaydı reddeder ve -1 döner. |
| `validator: şifre length >= 6` | Şifre güvenliği için en az 6 karakter zorunluluğu. |
| `contentPadding: symmetric(16, 15)` | Giriş alanlarının iç ferahlığını ve okunabilirliğini sağlar. |
| `prefixIcon: Icons.badge_rounded vb.` | Öğrenci no için kimlik, bölüm için okul ikonu ile zenginleştirme. |

## 6.3. Ana Ekran: Cüzdanım Sekmesi (Wallet Dashboard)




### Ekran Amacı ve Yapılan İşlemler:

Uygulamanın finans yönetim merkezidir. Üstte öğrenci karşılama ve cüzdan özeti kartı, ortada net bütçe bakiyesi (₺), FinTech bakiye gizleme göz ikonu, aylık harcama ilerleme çubuğu, hızlı aksiyon barı (Gelir/Gider Ekle, Analiz, Yenile) ve kaydırarak silinebilen (Dismissible) son işlem listesi yer alır.

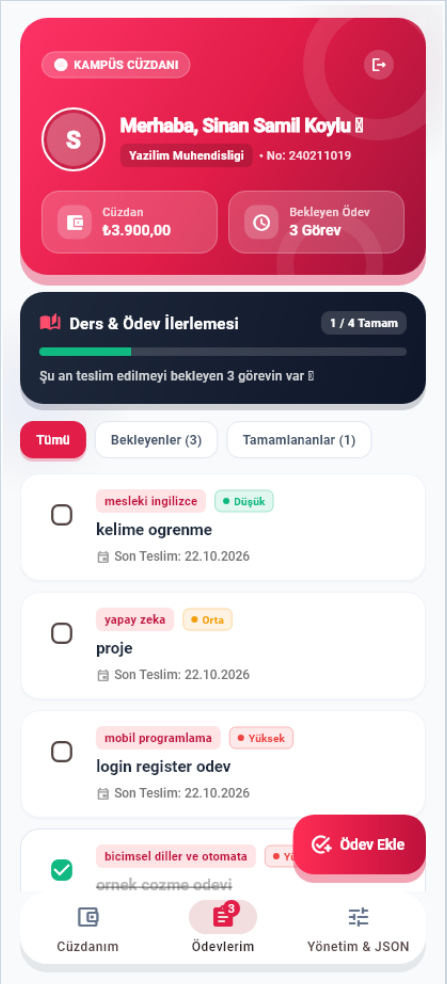
### Kullanılan Başlıca Widget'lar ve Rolleri:

| Widget | Kullanım Amacı |
|---|---|
| **Stack & Positioned (Glow)** | Arka plandaki şık radyal ışık parıltıları. |
| **LinearProgressIndicator** | Aylık harcamanın bütçeye oranını yüzde olarak dolan çubukla gösterme. |
| **IconButton (Bakiye Gizleme)** | Bakiyeyi '••••••' veya '₺3.900,00' olarak anlık değiştirme. |
| **Row & Action Buttons** | Gelir Ekle, Gider Ekle, Analiz ve Yenile hızlı erişim butonları. |
| **Dismissible & ListTile** | İşlemleri parmakla sağa/sola kaydırarak SQLite'tan silme jesti. |
| **FloatingActionButton** | Sağ altta degrade renkli '+ İşlem Ekle' yüzen butonu. |

### Öne Çıkan Property'ler ve Tercih Sebepleri:

| Property & Değeri | Neden Seçildi? (Teknik Gerekçe) |
|---|---|
| `LinearGradient: [Red, Rose, Crimson]` | Üst kartta ve FAB butonunda dikkat çekici FinTech renk geçişi. |
| `NumberFormat.currency(symbol: '₺')` | Rakamların Türk Lirası para birimi standardında biçimlendirilmesi. |
| `confirmDismiss & onDismissed` | Kaydırma ile SQLite'tan deleteTransaction sorgusu tetikleme. |
| `physics: AlwaysScrollableScrollPhysics` | İşlem sayısı az olsa da aşağı çekerek yenileme (pull-to-refresh). |

## 6.4. Ana Ekran: Ödevlerim Sekmesi (Task Manager)




### Ekran Amacı ve Yapılan İşlemler:

Akademik görev ve ödev takip merkezidir. Üstteki lacivert kartta tamamlanma yüzdesi barı (1/4 Tamam) ve kalan teslim sayısı gösterilir. Filtre çipleriyle (Tümü, Bekleyenler, Tamamlananlar) liste dinamik filtrelenir. Ödev kartlarındaki renkli öncelik etiketleri (Yüksek □, Orta □, Düşük □) aciliyeti belirtir.

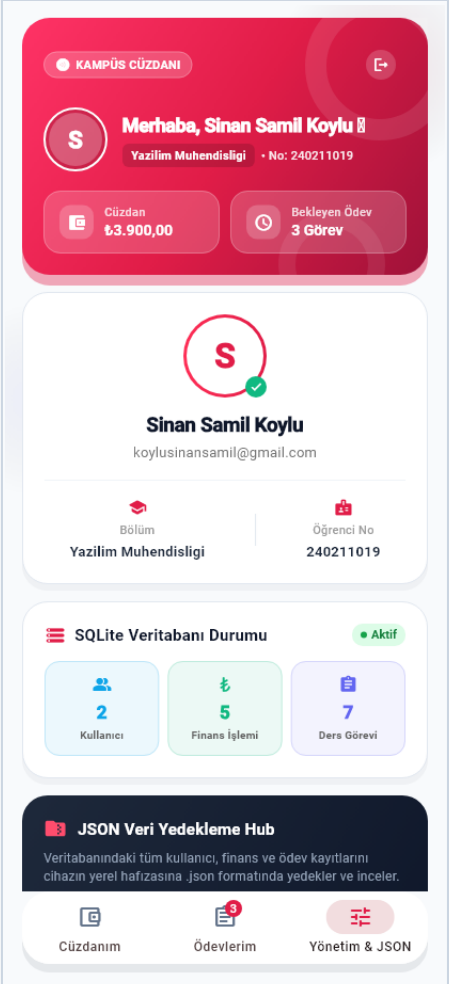
### Kullanılan Başlıca Widget'lar ve Rolleri:

| Widget | Kullanım Amacı |
|---|---|
| **Container (Koyu Lacivert Kart)** | Ders & Ödev ilerlemesini vurgulayan özel koyu tema özet kartı. |
| **LinearProgressIndicator** | Tamamlanan ödev oranını yeşil renkle dolduran ilerleme barı. |
| **ChoiceChip (3 Adet Filtre)** | Tümü, Bekleyenler (3) ve Tamamlananlar (1) anlık filtreleme butonları. |
| **Checkbox & ListTile** | Ödev tamamlandı durumunu (0/1) tek tıkla SQLite'ta güncelleyen kutu. |
| **Container (Öncelik Rozeti)** | Ödevin aciliyetine göre dinamik renklenen şık çip. |
| **FloatingActionButton** | Sağ altta yüzen '+ Ödev Ekle' butonu. |

### Öne Çıkan Property'ler ve Tercih Sebepleri:

| Property & Değeri | Neden Seçildi? (Teknik Gerekçe) |
|---|---|
| `is_completed: isCompleted ? 1 : 0` | SQLite'ta boolean değeri tamsayı bit olarak güncelleme. |
| `orderBy: 'is_completed ASC, id DESC'` | Tamamlanmamış acil ödevleri daima listenin en üstünde gösterme. |
| `valueColor: AlwaysStoppedAnimation` | Tamamlanma barını canlı zümrüt yeşili (#10B981) ile boyama. |
| `borderRadius: BorderRadius.circular(18)` | Ödev kartlarına yumuşak kenarlık ve hafif gölge verme. |

## 6.5. Ana Ekran: Yönetim & JSON Sekmesi (Profile & Hub)




### Ekran Amacı ve Yapılan İşlemler:

Öğrencinin profil bilgilerini (Ad Soyad, E-posta, Bölüm, Öğrenci No), SQLite veritabanının anlık durum istatistiklerini (toplam kullanıcı, işlem ve görev sayıları) ve tek dokunuşla tüm sistemi yedekleyen JSON Dışa Aktarma Hub'ını sunar. Güvenli oturum kapatma diyaloğu içerir.

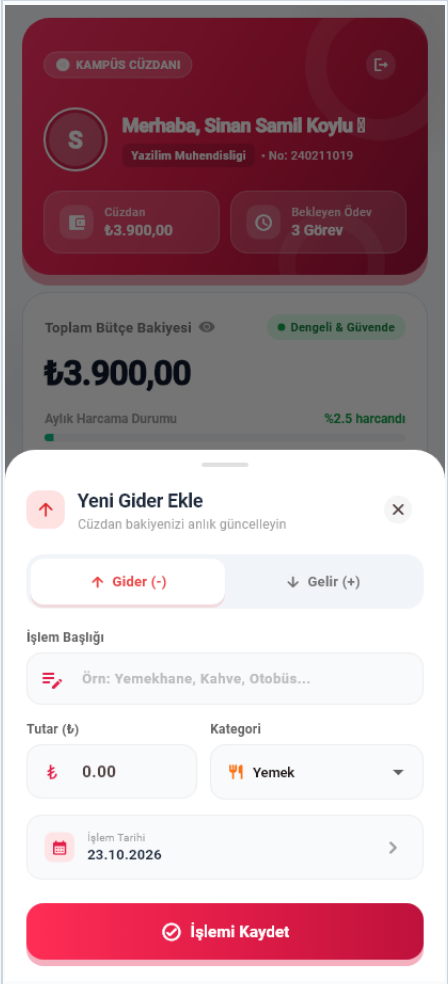
### Kullanılan Başlıca Widget'lar ve Rolleri:

| Widget | Kullanım Amacı |
|---|---|
| **CircleAvatar & Badge** | Öğrencinin adının baş harfini taşıyan ve yeşil onay rozeti olan büyük avatar. |
| **Container (İstatistik Kutuları)** | SQLite'tan canlı COUNT(*) sorgularıyla beslenen renkli sayaç hapları. |
| **Container (Siyah Degrade Hub)** | JSON dışa aktarma mekanizmasını barındıran koyu şık kart. |
| **ElevatedButton.icon** | JSON dosyasını üretip ekranda önizleme diyaloğunu açan buton. |
| **OutlinedButton (Çıkış Yap)** | Oturumu güvenle kapatmak için onay diyalogu tetikleyen buton. |

### Öne Çıkan Property'ler ve Tercih Sebepleri:

| Property & Değeri | Neden Seçildi? (Teknik Gerekçe) |
|---|---|
| `getDatabaseStats(): rawQuery('SELECT COUNT(*)')` | SQLite tablolarındaki toplam satır sayısını canlı okur. |
| `boxShadow: blurRadius 14, offset (0, 6)` | Kartlara temiz ve ferah derinlik hissi katar. |
| `AlertDialog: Çıkış Yapılsın mı?` | Yanlışlıkla çıkış yapılmasını engelleyen onay penceresi. |
| `Navigator.pushReplacement` | Çıkış yapılınca geçmişi temizleyip Login ekranına yönlendirir. |

## 6.6. Yeni İşlem Ekleme Modalı (Add Transaction Modal)




### Ekran Amacı ve Yapılan İşlemler:

Kullanıcının bütçesine yeni bir gelir veya gider kaydı eklemesini sağlar. Gider (-) ve Gelir (+) arasında tek tıkla geçiş yapılır. Tutar, başlık, kategori seçici ve sistem takviminden tarih seçimi (DatePicker) bileşenlerini barındırır.

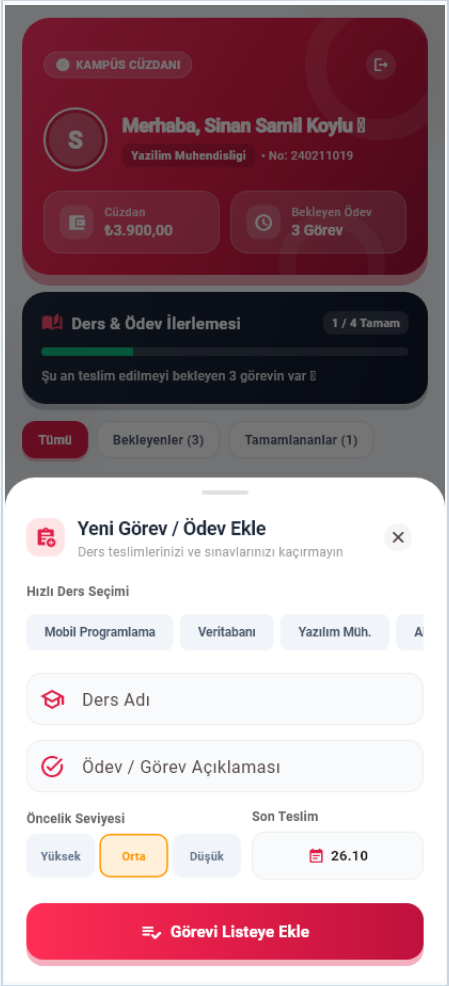
### Kullanılan Başlıca Widget'lar ve Rolleri:

| Widget | Kullanım Amacı |
|---|---|
| **showModalBottomSheet** | Alttan açılan şık yuvarlatılmış (28px) pencere. |
| **AnimatedContainer (Tür Seçici)** | Gider ve Gelir arasında pürüzsüz geçiş sağlayan buton çubuğu. |
| **TextFormField (Tutar ve Başlık)** | İşlem adı ve ₺ tutarı girişi. |
| **DropdownButtonFormField** | Yemek, Ulaşım, Kitap, Burs gibi kategorileri ikonlarıyla sunan seçici. |
| **InkWell & showDatePicker** | İşlem tarihini takvimden seçtiren interaktif tarih kutusu. |
| **ElevatedButton (Kaydet)** | İşlemi SQLite transactions tablosuna yazıp cüzdanı yenileyen buton. |

### Öne Çıkan Property'ler ve Tercih Sebepleri:

| Property & Değeri | Neden Seçildi? (Teknik Gerekçe) |
|---|---|
| `isScrollControlled: true` | Modalın klavye açıldığında yukarı esnemesini sağlar. |
| `firstDate: 2025, lastDate: 2030` | DatePicker için geçerli tarih aralığı tanımlaması. |
| `amount.replaceAll(',', '.')` | Türkçe virgüllü ondalık sayıları double veri tipine dönüştürme. |
| `Navigator.pop(context, true)` | Kayıt başarılıysa ana ekrana 'true' dönerek anlık yenileme tetikler. |

## 6.7. Yeni Ödev Ekleme Modalı (Add Task Modal)




### Ekran Amacı ve Yapılan İşlemler:

Öğrencinin yeni bir ders görevi veya sınav teslimi oluşturmasını sağlar. Ders adı için hızlı ders seçim çipleri (Mobil Programlama, Veritabanı, Yazılım Müh.) sunar. Öncelik derecesi (Yüksek, Orta, Düşük) ve son teslim tarihi takvim seçicisi içerir.

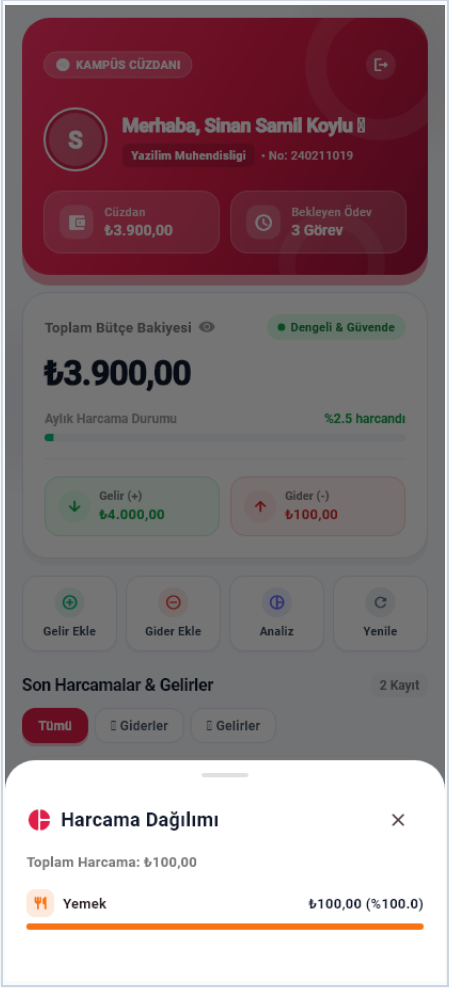
### Kullanılan Başlıca Widget'lar ve Rolleri:

| Widget | Kullanım Amacı |
|---|---|
| **showModalBottomSheet** | Yuvarlatılmış üst köşelere sahip alttan açılır pencere. |
| **SingleChildScrollView** | Ekran boyutu küçük cihazlarda formun kaydırılabilmesini sağlar. |
| **ActionChip (Hızlı Ders Çipleri)** | Ders adını tek dokunuşla TextFormField'a kopyalayan butonlar. |
| **ChoiceChip (Öncelik Düzeyi)** | Yüksek, Orta, Düşük önceliklerini renkli butonlarla seçtiren yapı. |
| **InkWell & showDatePicker** | Son teslim tarihini takvim diyaloğundan seçtiren bileşen. |
| **ElevatedButton (Görevi Listeye Ekle)** | Görevi tasks tablosuna ekleyen birincil buton. |

### Öne Çıkan Property'ler ve Tercih Sebepleri:

| Property & Değeri | Neden Seçildi? (Teknik Gerekçe) |
|---|---|
| `initialDate: now.add(Duration(days: 3))` | Varsayılan teslim tarihini 3 gün sonraya ayarlama. |
| `selectedColor: Color(0xFFFEF3C7)` | Seçili öncelik çipine canlı arka plan vurgusu verme. |
| `priority: 'Yüksek' \| 'Orta' \| 'Düşük'` | Ödevin önem derecesini metinsel olarak saklama. |
| `isCompleted: false` | Yeni eklenen görevlerin varsayılan olarak 'bekliyor' durumunda açılması. |

## 6.8. Kategori Harcama Dağılım Modalı (Category Analysis)




### Ekran Amacı ve Yapılan İşlemler:

Kullanıcının yaptığı tüm harcamaları kategorilerine göre gruplayarak (Yemek, Ulaşım, Kitap vb.) toplam harcama tutarlarını ve harcama içindeki yüzdesel oranlarını (%100.0) renkli ilerleme çubuklarıyla analitik olarak görselleştirir. Öğrencinin parasını nereye harcadığını net görmesini sağlar.

### Kullanılan Başlıca Widget'lar ve Rolleri:

| Widget | Kullanım Amacı |
|---|---|
| **showModalBottomSheet** | Alttan açılan detaylı analiz penceresi. |
| **Column & Row (Dinamik Liste)** | Kategori haritalamasındaki (Map) verileri dönerek liste oluşturma. |
| **LinearProgressIndicator** | Kategorinin toplam harcamadaki yüzdesini gösteren renkli çubuk. |
| **Icon & Container** | Her kategoriye özel renk ve simge kutusu (Yemek için çatal-bıçak, Ulaşım için otobüs). |
| **IconButton (Kapat)** | Modaldan çıkışı sağlayan çarpı ikonu. |

### Öne Çıkan Property'ler ve Tercih Sebepleri:

| Property & Değeri | Neden Seçildi? (Teknik Gerekçe) |
|---|---|
| `ratio = entry.value / _totalExpense` | Kategorinin toplam harcama içindeki payını hesaplama (0.0 - 1.0). |
| `toStringAsFixed(1)` | Harcama yüzdesini virgülden sonra 1 basamakla (%25.4) biçimlendirme. |
| `categoryMap.entries.map(...)` | SQL'den gelen verileri dinamik grafik barlarına haritalama. |
| `valueColor: AlwaysStoppedAnimation(catColor)` | Her kategoriye kendine has pastel renk atama. |

## 6.9. JSON Dışa Aktarma & Kod Önizleme Diyaloğu




### Ekran Amacı ve Yapılan İşlemler:

SQLite veritabanındaki tüm tabloları (users, transactions, tasks) ilişkisel olarak çekip standart .json formatında cihazın yerel depolama dizinine kaydeder (ogrenci_asistani_yedek.json). Ekranda açılan koyu renkli terminal penceresinde JSON içeriğini renkli monospace fontla önizletir.

### Kullanılan Başlıca Widget'lar ve Rolleri:

| Widget | Kullanım Amacı |
|---|---|
| **showDialog & AlertDialog** | Merkezi açılır sistem diyaloğu. |
| **Container (Terminal Penceresi)** | Koyu lacivert (#0F172A) arka planlı kod kutusu. |
| **SingleChildScrollView** | Yüzlerce satırlık JSON metnini kutu içinde akıcı kaydırma. |
| **Text (Monospace Syntax)** | Consolas / Monospace yazı tipiyle kod renklendirmesi. |
| **ElevatedButton (Kapat)** | Önizleme diyaloğunu sonlandıran onay butonu. |

### Öne Çıkan Property'ler ve Tercih Sebepleri:

| Property & Değeri | Neden Seçildi? (Teknik Gerekçe) |
|---|---|
| `JsonEncoder.withIndent(' ')` | JSON verisini 2 boşluk girintili, okunabilir formatta serileştirme. |
| `getApplicationDocumentsDirectory()` | Dosyayı işletim sisteminin güvenli yerel hafızasına kaydetme. |
| `fontFamily: 'monospace'` | Kodların gerçek bir IDE terminalindeymiş gibi hizalı gösterimi. |
| `color: Color(0xFF38BDF8)` | JSON metnini göz yormayan açık mavi/cyan renkle vurgulama. |

## 7. Güvenlik, Veri Doğrulama ve Test Mekanizmaları

Projeyi geliştirirken yalnızca görsel arayüze değil, arka plandaki yazılım güvenliği, veri bütünlüğü ve otomatik test edilebilirlik standartlarına azami özen gösterilmiştir:

| Güvenlik / Test Alanı | Uygulanan Yöntem & Kütüphane | Sağlanan Koruma ve Avantaj |
|---|---|---|
| **SQL Enjeksiyon Koruması (SQL Injection Defense)** | sqflite parametrik sorguları whereArgs: [email, password] | Kullanıcı girdileri asla ham SQL dizesiyle birleştirilmez (string concatenation yapılmaz). Bu sayede tırnak işareti veya özel karakterlerle SQL açığı oluşturulması %100 engellenir. |
| **İlişkisel Veri Bütünlüğü (Cascade Deletion)** | SQLite PRAGMA foreign_keys = ON & ON DELETE CASCADE | Kullanıcı hesabı silindiğinde yetim veri (orphaned data) kalmaz; kullanıcıya ait tüm gelir, gider ve ders görevleri ilişkisel olarak veritabanı seviyesinde temizlenir. |
| **Form Validasyon Motoru (Input Validation)** | GlobalKey<FormState> & RegExp Kontrolleri | Geçersiz e-posta formatları (@ ve domain eksikliği), 6 karakterden kısa şifreler veya sıfırdan küçük harcama tutarları daha veritabanına ulaşmadan arayüzde yakalanarak engellenir. |
| **Otomatik Widget Testleri (Automated Testing)** | flutter_test & RenderRepaintBoundary toImage | Uygulamanın bileşenleri, mock SharedPreferences ve FFI veritabanı ile otomatik testlerden geçirilmiş; duman testleri (smoke tests) ve ekran yakalama testleri başarıyla koşturulmuştur. |

## 8. Sonuç, Öğrenim Kazanımları ve Değerlendirme

Bu proje kapsamında modern mobil yazılım geliştirme ekosisteminin lider teknolojisi olan Flutter & Dart kullanılarak, gerçek dünya problemlerini çözen, katmanlı mimariye (Layered Architecture) sahip, veri kalıcılığı yerel SQLite ve JSON File I/O ile sağlanan, kullanıcı deneyimi üst seviyede bir mobil uygulama başarıyla ortaya konulmuştur.

Elde Edilen Temel Mühendislik Kazanımları:
1. Flutter Widget Yaşam Döngüsü: StatelessWidget ve StatefulWidget arasındaki farklar, `initState`, `build`, `dispose` metotları ve `setState` üzerinden reaktif UI güncelleme mantığı derinlemesine kavranmıştır.
2. İlişkisel Veritabanı ve CRUD İşlemleri: SQLite üzerinde CREATE TABLE, INSERT, SELECT, UPDATE, DELETE komutları; yabancı anahtar (Foreign Key) kısıtlamaları ve aggregate (SUM, COUNT) sorguları pratik edilmiştir.
3. Dosya Sistemi ve JSON Serileştirme: Dart'ın `dart:convert` kütüphanesiyle nesne-JSON dönüşümleri ve `path_provider` aracılığıyla cihazın yerel depolama alanına fiziksel dosya yazma/okuma operasyonları tamamlanmıştır.
4. FinTech ve Material 3 Tasarım Standartları: Degrade geçişler, bakiye gizleme mekanizması, ilerleme çubukları, animasyonlu kaydırmalı silme (Dismissible) ve alt navigasyon dock'u ile profesyonel bir arayüz tecrübesi kazanılmıştır.

Sonuç olarak, Öğrenci Asistanı projesi hem kod kalitesi, hem veri güvenliği, hem de arayüz zarafeti açısından üniversite Mobil Programlama dersinin tüm teknik ve işlevsel kazanımlarını eksiksiz olarak karşılamaktadır.